

setTimeout and EventListener asynchronous -> Here comes the concept of WebAPI

Synchronize means jis order me diya hai usi order me execute karnataka

Koi bhi javascript ka code call stack ke andhar aayega and GEC banke execute hogaa and execute hokar GEC hat jaayega

Javascript KABHI BHI KISI CHI KA WAIT NHI KAREGA , wo statement ko dekhega execute karke aage niklega, setTimeout ko bolega tu jaayega bhaadme kyuki 5 sec nahi wait karega wo

It is like execute agla execute agla execute aglaa , wo aapke code ko block hone nahi dega

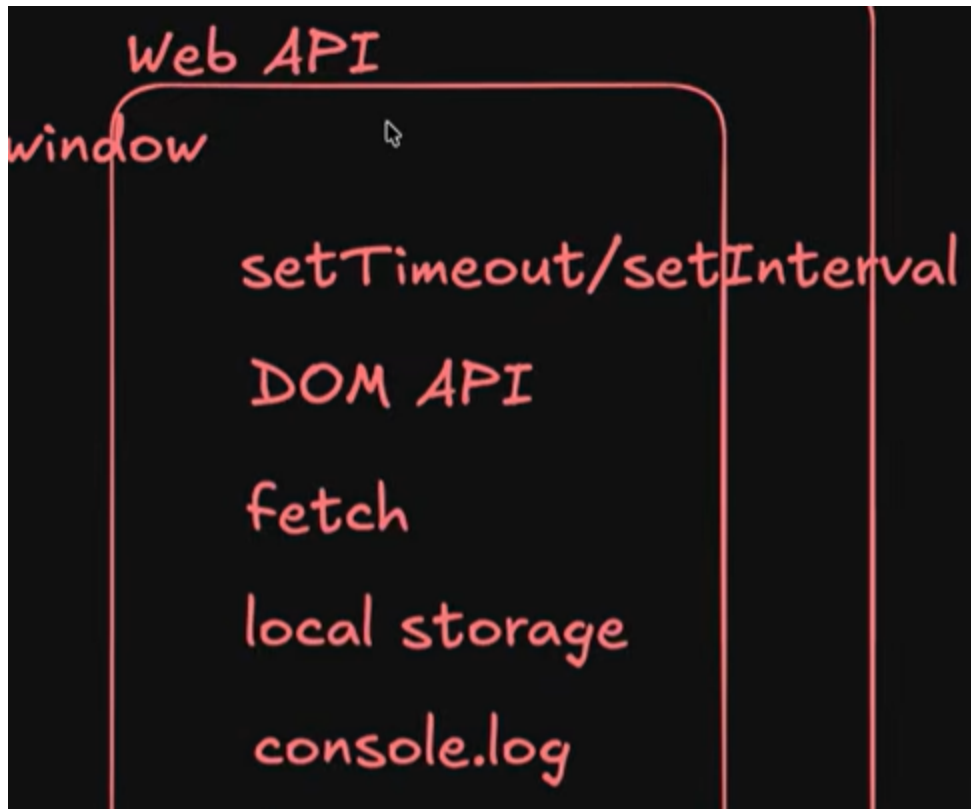
API is nothing but a function call , aap function call kar rahe hai and wahase result aa raha hai without knowing how it is working internally

Dusri type ki api hoti jisse hum get req, post req, dekho bhai wo bhi function call hoti hai

Window object hi webapi ka access deta hai

Window object aapka browser create karta hai and browser hi aapko kuch code ka access deta hai

Webapis aapko ye sab access dete hai



JavaScript single threaded but Browser is multithreaded

Javascript says setTimeout ko me sambhal nahi sakta toh jabaabdaari window object ki hai jo webapis hai jo browser handle karta hai

Browser callback function ko pakdega , jaise hi 5 sec ho jaayenge , notice degaa ki iss code ko execute kar denaaa

Browser has timer access , jaise hi 5 sec khatam , wo callback queue me daal dega
It has register

Event Loop hai sabka Daddy

JavaScript work is jo bhi callstack ke andhar hai , unko execute karna

Event loop ka kaam hi hai ki callback queue hamesha check karte raho , and agar hai toh call stack me daala with the condition ki woh khaali hona chahiye so that predictable call aa jaaye

Completed till 32:50

add EventListener me WebAPI listen karega , kyukii wo uskepaas hai

Lets say button 1 ko click kiya , webapi listened it , pushed the callback function to callback queue and then event loop if the callstack is empty then push it , jaise hi stack pe aayega , javascript knows ki jo bhi call stack me aayega execute it .

JavaScript has one principle , I will wait for no one , but whenever waiting , asynchronous task aayega , I will assign it to WebAPI and agar event occur hota hai toh mujhe bata diyo

Microtask Queue has high priority than Callback queue

Fetch waala operation , jahase aap data laa rahe hotey hai , wo iss queue me jaatey hai

Settimeout and setInterval , event listener , unke callback function callback queue me jaatey hai

JavaScript Execution Model: Complete In-Depth Guide

Table of Contents

1. [Core Concepts](#)
 2. [Single-Threaded Nature](#)
 3. [Synchronous Execution](#)
 4. [The Call Stack](#)
 5. [Web APIs](#)
 6. [Asynchronous Behavior](#)
 7. [Callback Queue](#)
 8. [Microtask Queue](#)
 9. [Event Loop](#)
 10. [Practical Examples](#)
-

Core Concepts

JavaScript is a Single-Threaded Synchronous Language

JavaScript fundamentally operates on two key principles:

- **Single-Threaded:** Only one task executes at a time
- **Synchronous:** Tasks execute in order, one after another

However, JavaScript can exhibit **asynchronous behavior** through the help of Web APIs and the browser environment.

Single-Threaded Nature

What Does Single-Threaded Mean?

Definition: Only one task can execute at any given moment. Multiple tasks cannot run in parallel.

Example:

```
console.log("First");  
console.log("Second");  
console.log("Third");
```

Execution Flow:

1. Prints "First"
2. Then prints "Second"
3. Finally prints "Third"

Key Point: These statements do NOT execute in parallel. Each must complete before the next begins.

The Problem with Blocking Code

Consider this example:

```
console.log("First");  
  
let sum = 0;  
for(let i = 0; i < 1000000000; i++) {
```

```
    sum += i;
}
console.log(sum);

console.log("Last");
```

What Happens:

1. "First" prints immediately
2. The loop takes significant time to complete
3. During this time, the entire page becomes **unresponsive**
4. Buttons stop working
5. No other JavaScript can execute
6. Only after the loop completes does "Last" print

Why?: JavaScript is single-threaded. While calculating the sum, it cannot do anything else. The entire execution thread is blocked.

Real-World Impact

During heavy computations:

- UI freezes
- Buttons become unclickable
- User interactions are ignored
- The page appears "hung"

This demonstrates why JavaScript needs mechanisms to handle long-running operations without blocking.

Synchronous Execution

What Does Synchronous Mean?

Definition: Code executes in the exact order written, step-by-step, without skipping ahead.

Synchronous Flow:

```
console.log("Step 1");
console.log("Step 2");
console.log("Step 3");
```

Execution: Step 1 → Step 2 → Step 3 (in perfect sequence)

NOT: Step 1 → Step 3 → Step 2 (this would be out of order)

The Principle:

- JavaScript reads each line
 - Executes it completely
 - Moves to the next line
 - Never skips ahead
 - Never goes back (unless explicitly told via loops/functions)
-

The Call Stack

What is the Call Stack?

The **Call Stack** is a data structure that manages code execution in JavaScript. It follows the **LIFO** (Last In, First Out) principle.

How It Works:

```
console.log("1");  
console.log("2");  
console.log("3");  
console.log("4");
```

Stack Visualization:

Step 1: [Global Execution Context] - added
Step 2: Execute console.log("1") - prints "1"
Step 3: Execute console.log("2") - prints "2"
Step 4: Execute console.log("3") - prints "3"
Step 5: Execute console.log("4") - prints "4"
Step 6: [Global Execution Context] - removed

Global Execution Context

When JavaScript code runs:

1. A **Global Execution Context** is created
2. All code is placed inside it

3. Code executes line by line
4. When complete, the context is removed from the stack

Key Characteristics:

- **LIFO**: Last In, First Out
 - **One at a time**: Only the top item executes
 - **Blocking**: If one operation takes time, everything waits
 - **Synchronous**: Maintains execution order
-

Web APIs

What are Web APIs?

Critical Understanding: Web APIs are NOT part of JavaScript itself. They are provided by the browser environment.

The Simple Truth About APIs

API = Function Call

An API is simply calling a function where:

- You don't know (or need to know) the internal implementation
- You just use it to get results
- Someone else handles the complex work

Examples of API Calls:

// Array methods - these are API calls

```
let arr = [1, 2, 3];
```

```
arr.push(10); // API call - don't know internal implementation
```

```
arr.map(x => x * 10); // API call
```

```
arr.sort(); // API call
```

// Custom function - also an API call

```
function hello() {  
  console.log("Hi");  
}
```

```
hello(); // API call to your own function
```

// Console - Web API call

```
console.log("Hello"); // Don't know how it displays on screen
```

The Window Object

The browser creates a **global Window object** that provides access to Web APIs:

```
// These are all part of the Window object
window.setTimeout();
window.setInterval();
window.fetch();
window.localStorage;
window.document;
window.console;
```

What Web APIs Provide:

1. **setTimeout / setInterval**: Timer functionality
2. **DOM APIs**: Document manipulation
 - `document.getElementById()`
 - `element.addEventListener()`
3. **Fetch API**: Network requests
4. **Console API**: Logging to console
5. **Local Storage**: Browser storage
6. **Location API**: URL information
7. And many more...

Critical Distinction:

- **JavaScript**: Single-threaded, synchronous
- **Browser/Web APIs**: Multi-threaded, can handle multiple operations simultaneously

The Power of Web APIs:

Web APIs have capabilities JavaScript lacks:

- **Built-in timers**: Can count time independently
- **Event listening**: Can monitor multiple events simultaneously
- **Network operations**: Can make HTTP requests
- **Multi-threading**: Can handle multiple operations in parallel

Asynchronous Behavior

The Apparent Contradiction

Question: If JavaScript is single-threaded and synchronous, how can it handle asynchronous operations?

Answer: It can't. But the **browser** can.

Example 1: setTimeout

```
console.log("Hello");

setTimeout(() => {
  console.log("Timeout Executed");
}, 5000);

console.log("I am the end");
```

Output:

```
Hello
I am the end
(wait 5 seconds)
Timeout Executed
```

The Mystery:

1. Why didn't JavaScript wait for 5 seconds before printing "I am the end"?
2. Who was counting the 5 seconds if JavaScript was already executing the next line?
3. How is the timer running while other code executes?

The Answer: Web APIs Handle It

Execution Flow:

1. JavaScript encounters setTimeout
2. JavaScript says: "I don't handle timers"
3. Passes setTimeout to Web API with:
 - The callback function
 - The time delay (5000ms)
4. JavaScript immediately moves to next line
5. Web API starts counting the time in parallel
6. JavaScript continues executing remaining code
7. When 5 seconds complete, Web API notifies JavaScript
8. Callback function gets added to Callback Queue

9. Event Loop moves it to Call Stack when stack is empty
10. Finally, callback executes

JavaScript's Philosophy:

"I don't wait for anyone"

- JavaScript never blocks
- If something takes time → delegate to Web API
- Keep executing the next lines
- Come back to delayed tasks later

Example 2: Multiple Timers

```
console.log("Hello");
```

```
setTimeout(() => {  
  console.log("Timeout 1");  
}, 5000);
```

```
setTimeout(() => {  
  console.log("Timeout 2");  
}, 6000);
```

```
console.log("I am the end");
```

Output:

Hello

I am the end

(after 5 seconds) Timeout 1

(after 1 more second) Timeout 2

How are both timers running simultaneously?

Because Web APIs are **multi-threaded**. They can:

- Count Timer 1 (5 seconds)
- Count Timer 2 (6 seconds)
- Both at the same time!

JavaScript couldn't do this alone.

Example 3: Event Listeners

```
console.log("Hi");

button1.addEventListener('click', () => {
  console.log("Button 1 clicked");
});

button2.addEventListener('click', () => {
  console.log("Button 2 clicked");
});

button3.addEventListener('click', () => {
  console.log("Button 3 clicked");
});

console.log("I am the end");
```

Question: How is JavaScript listening to all three buttons simultaneously?

Answer: It's not. The **Web API** is.

Process:

1. JavaScript encounters first `addEventListener`
2. Passes it to Web API: "Monitor button1 for clicks"
3. Immediately moves to next line
4. JavaScript encounters second `addEventListener`
5. Passes it to Web API: "Monitor button2 for clicks"
6. Same for button3
7. JavaScript finishes executing all code
8. Web API is now monitoring all three buttons in parallel
9. When any button is clicked, Web API adds the callback to Callback Queue
10. Event Loop moves it to Call Stack for execution

Why This Matters:

Without Web APIs, JavaScript would have to:

- Check button1 for click
- Then check button2
- Then check button3
- Repeat forever

Meanwhile, nothing else could execute. The page would be frozen.

Callback Queue

What is the Callback Queue?

A **queue** (First In, First Out) that holds callback functions waiting to be executed.

How Callbacks Enter the Queue:

From `setTimeout`:

```
setTimeout(() => {  
  console.log("Callback");  
}, 5000);
```

When 5 seconds complete:

1. Web API takes the callback function
2. Places it in the Callback Queue
3. Waits for Event Loop to process it

From Event Listeners:

```
button.addEventListener('click', () => {  
  console.log("Clicked");  
});
```

When button is clicked:

1. Web API detects the click
2. Takes the callback function
3. Places it in the Callback Queue

Queue Characteristics:

- **FIFO**: First In, First Out
- **Order matters**: Callbacks execute in the order they arrive
- **Wait for empty stack**: Callbacks only move to Call Stack when it's empty

Example Flow:

Callback Queue: [Callback1] [Callback2] [Callback3]

↑ Next to execute

Callback1 goes to Call Stack first, then Callback2, then Callback3.

Microtask Queue

What is the Microtask Queue?

A special queue with **higher priority** than the Callback Queue.

Priority System:

Microtask Queue: HIGH PRIORITY

Callback Queue: NORMAL PRIORITY

What Goes Where?

Microtask Queue:

- Promises (`.then()`, `.catch()`, `.finally()`)
- Fetch API responses
- `queueMicrotask()`
- MutationObserver callbacks

Callback Queue:

- `setTimeout` callbacks
- `setInterval` callbacks
- Event listeners (click, scroll, etc.)
- I/O operations

Example:

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Timeout");  
}, 0);
```

```
fetch("https://api.github.com/users")  
  .then(() => {
```

```
    console.log("Fetch complete");  
  });  
  
console.log("End");
```

Output:

Start
End
Fetch complete
Timeout

Why?

1. "Start" and "End" execute immediately (synchronous)
2. Both setTimeout and fetch register with Web APIs
3. Both complete almost simultaneously
4. Fetch callback goes to **Microtask Queue**
5. setTimeout callback goes to **Callback Queue**
6. Event Loop checks **Microtask Queue first**
7. "Fetch complete" executes before "Timeout"

The Rule:

Event Loop always prioritizes:

1. Current Call Stack (must be empty)
2. Microtask Queue (checked first)
3. Callback Queue (checked second)

Event Loop

What is the Event Loop?

The **Event Loop** is the mechanism that coordinates between the Call Stack, Callback Queue, and Microtask Queue.

The Event Loop's Job:

Continuously ask two questions:

1. "Is there anything in the Microtask Queue?"
2. "Is the Call Stack empty?"

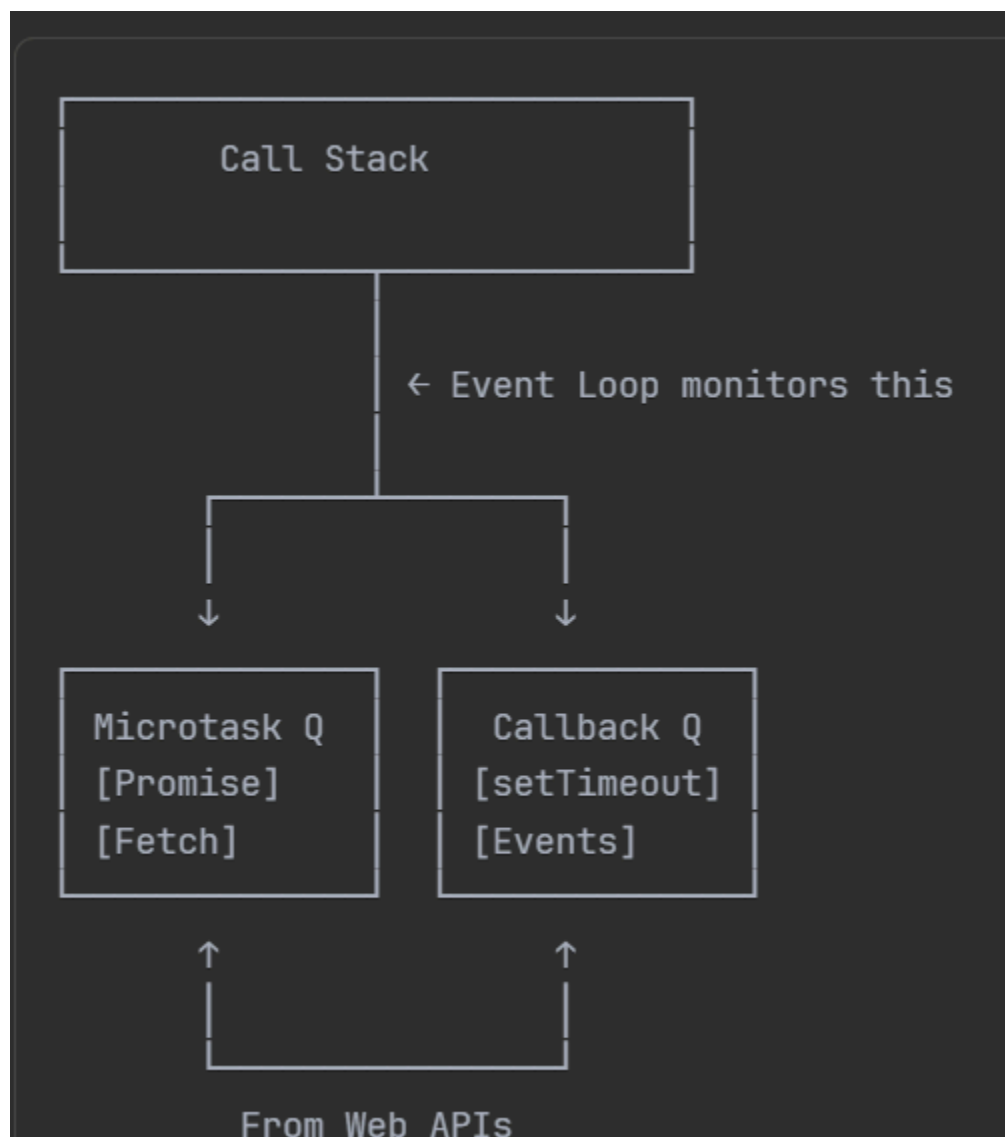
If **both conditions are true**:

- Take a callback from Microtask Queue
- Place it on the Call Stack
- Let JavaScript execute it

If Microtask Queue is empty:

- Check Callback Queue
- Do the same process

Visual Representation:



The Algorithm:

```
while (true) {  
  if (callStack.isEmpty()) {  
    if (microtaskQueue.hasCallbacks()) {  
      callback = microtaskQueue.dequeue();  
      callStack.push(callback);  
    } else if (callbackQueue.hasCallbacks()) {  
      callback = callbackQueue.dequeue();  
      callStack.push(callback);  
    }  
  }  
}
```

Why Stack Must Be Empty:

Reason: Predictable execution order.

Scenario Without This Rule:

```
console.log("1");  
console.log("2");  
console.log("3");  
console.log("4");
```

If a callback could interrupt:

- Sometimes callback executes between "2" and "3"
- Sometimes between "3" and "4"
- Unpredictable output!

With Empty Stack Rule:

- All synchronous code completes first
- Then callbacks execute
- Predictable, consistent behavior

Example:

```
console.log("1");  
  
setTimeout(() => {
```

```
    console.log("Timeout");  
  }, 0);
```

```
console.log("2");  
console.log("3");  
console.log("4");
```

Output:

```
1  
2  
3  
4  
Timeout
```

Even with 0ms delay, "Timeout" prints last because:

1. Synchronous code executes first (1, 2, 3, 4)
 2. Call Stack empties
 3. Event Loop moves setTimeout callback to stack
 4. "Timeout" executes
-

Practical Examples

Example 1: Complete setTimeout Flow

```
console.log("Hello");
```

```
setTimeout(() => {  
  console.log("Timeout Executed");  
}, 5000);
```

```
console.log("I am the end");
```

Step-by-Step Execution:

1. **Global Execution Context created** → Added to Call Stack
2. **Line 1:** `console.log("Hello")`
 - Web API prints "Hello"
 - Continues to next line

3. **Line 3:** `setTimeout(...)`
 - JavaScript sees this is async
 - Passes to Web API with:
 - Callback function
 - 5000ms delay
 - **Does NOT wait**
 - Immediately moves to next line
4. **Line 7:** `console.log("I am the end")`
 - Web API prints "I am the end"
 - All synchronous code complete
5. **Global Execution Context removed** → Call Stack now empty
6. **Meanwhile, Web API counting:** 1s, 2s, 3s, 4s, 5s...
7. **After 5 seconds:** Web API adds callback to Callback Queue
8. **Event Loop checks:**
 - Is Call Stack empty? ✓ Yes
 - Is there something in queue? ✓ Yes
9. **Event Loop moves callback** to Call Stack
10. **Callback executes:** "Timeout Executed" prints
11. **Callback removed** from Call Stack

Example 2: Multiple Event Listeners

```
console.log("Hi");
```

```
button1.addEventListener('click', () => {  
  console.log("Button 1 clicked");  
});
```

```
button2.addEventListener('click', () => {  
  console.log("Button 2 clicked");  
});
```

```
button3.addEventListener('click', () => {  
  console.log("Button 3 clicked");  
});
```

```
console.log("I am the end");
```

Registration Phase:

1. `console.log("Hi")` → Executes immediately
2. `button1.addEventListener(...)`
 - JavaScript: "This is async event monitoring"
 - Passes to Web API

- Web API registers: "Monitor button1, callback = callback1"
- 3. `button2.addEventListener(...)`
 - Passed to Web API
 - Registers: "Monitor button2, callback = callback2"
- 4. `button3.addEventListener(...)`
 - Passed to Web API
 - Registers: "Monitor button3, callback = callback3"
- 5. `console.log("I am the end")` → Executes immediately
- 6. Call Stack empties

Runtime Phase:

Web API now actively monitoring all three buttons simultaneously (because Web APIs are multi-threaded).

When User Clicks Button2:

1. Web API detects click on button2
2. Takes callback2
3. Adds to Callback Queue: `[callback2]`
4. Event Loop sees:
 - Stack empty? ✓
 - Queue has callback? ✓
5. Moves callback2 to Call Stack
6. Executes: "Button 2 clicked" prints

If User Quickly Clicks Button1, Then Button3:

1. Callback Queue: `[callback1, callback3]`
2. Event Loop processes callback1 first (FIFO)
3. "Button 1 clicked" prints
4. Then callback3 processes
5. "Button 3 clicked" prints

Example 3: Fetch API

```
console.log("Start the operation");
```

```
fetch('https://api.github.com/users')
  .then((response) => {
    console.log("GitHub user info");
  });
```

```
console.log("End of operation");
```

Execution Flow:

1. `console.log("Start the operation")` → Prints immediately
2. `fetch(...)`
 - JavaScript: "This is async network request"
 - Passes to Web API with:
 - URL to fetch from
 - Callback (the `.then()` function)
 - **Does NOT wait for response**
 - Moves to next line immediately
3. `console.log("End of operation")` → Prints immediately
4. Call Stack empties
5. **Meanwhile:** Web API making HTTP request to GitHub
6. **Response arrives** (could be 1s, 2s, 5s later)
7. Web API adds callback to **Microtask Queue** (not Callback Queue!)
8. Event Loop:
 - Stack empty? ✓
 - Microtask Queue has item? ✓
9. Moves callback to Call Stack
10. Callback executes: "GitHub user info" prints

Output:

Start the operation
End of operation
GitHub user info

Example 4: Mixed Async Operations

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Output:

Start
End
Promise
Timeout

Why This Order?

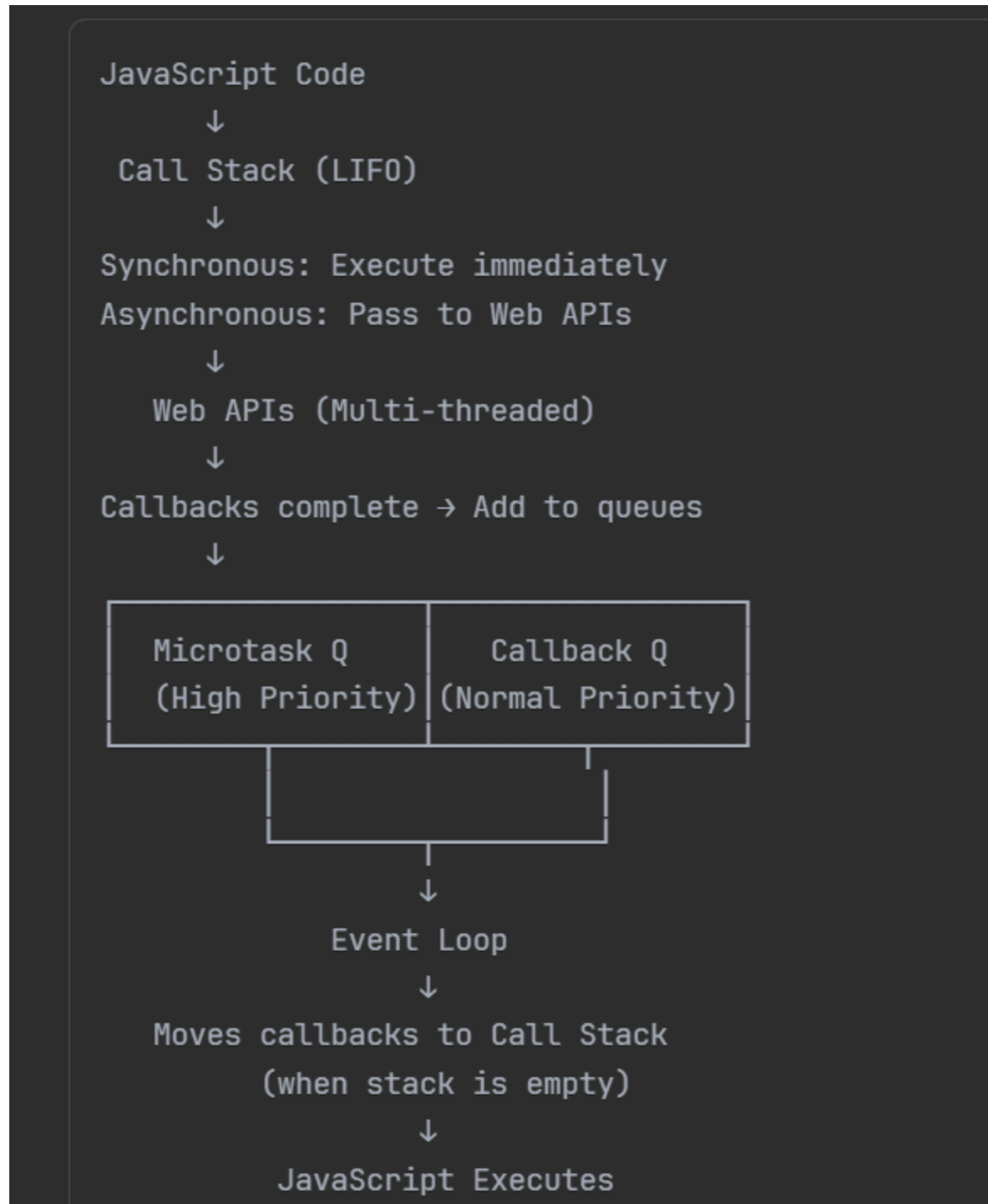
1. "Start" - synchronous
 2. setTimeout → Callback Queue
 3. Promise → Microtask Queue
 4. "End" - synchronous
 5. Stack empties
 6. Event Loop checks **Microtask Queue first**
 7. "Promise" executes
 8. Then checks Callback Queue
 9. "Timeout" executes
-

Key Takeaways

JavaScript's Core Principles:

1. **Single-Threaded**: One operation at a time
2. **Synchronous**: Executes in order
3. **Non-Blocking**: Never waits for async operations
4. **Delegates**: Passes async work to Web APIs

The Execution Model:



The Event Loop Rule:

Priority Order:

1. Call Stack must be empty

2. Check Microtask Queue first
3. Then check Callback Queue
4. Execute one callback at a time
5. Repeat forever

Why This Design?

- **Prevents Freezing:** Async operations don't block UI
 - **Maintains Order:** Predictable execution sequence
 - **Enables Responsiveness:** UI stays interactive
 - **Efficient:** Browser handles heavy lifting
 - **Simple API:** Developers just write callbacks
-

Common Misconceptions

✗ "JavaScript is asynchronous"

✓ **Truth:** JavaScript is synchronous. The **browser environment** provides asynchronous capabilities through Web APIs.

✗ "setTimeout guarantees execution after X milliseconds"

✓ **Truth:** setTimeout guarantees execution **at least** X milliseconds later, but could be much longer if the Call Stack is busy.

✗ "Callbacks execute immediately when their condition is met"

✓ **Truth:** Callbacks go to a queue and wait for the Call Stack to empty.

✗ "Event Loop is part of JavaScript"

✓ **Truth:** Event Loop is part of the browser's runtime environment, not the JavaScript language itself.

✗ "All callbacks are treated equally"

✓ **Truth:** Microtask Queue (Promises, fetch) has higher priority than Callback Queue (setTimeout, events).

Homework Questions - ANSWERED

Question 1: What other operations go into the Microtask Queue besides Promises and fetch?

Answer: Several operations go into the Microtask Queue:

1. Promise-related callbacks

```
Promise.resolve().then(() => {});    // Microtask
Promise.reject().catch(() => {});    // Microtask
Promise.finally(() => {});           // Microtask
```

2. queueMicrotask()

```
queueMicrotask(() => {
  console.log("Explicit microtask");
});
```

This is a direct way to add something to the Microtask Queue.

3. MutationObserver

```
const observer = new MutationObserver((mutations) => {
  console.log("DOM changed"); // This runs as a microtask
});
```

```
observer.observe(document.body, {
  childList: true,
  subtree: true
});
```

4. async/await (behind the scenes uses Promises)

```
async function example() {
  await somePromise(); // Microtask
  console.log("After await"); // This also becomes a microtask
}
```

5. process.nextTick() (Node.js only, not browser)

```
process.nextTick(() => {
  console.log("Next tick");
});
```

Microtask Queue Summary:

- All Promise handlers (`.then`, `.catch`, `.finally`)
 - `queueMicrotask()`
 - MutationObserver callbacks
 - Async/await continuations
 - `process.nextTick()` in Node.js
-

Question 2: What happens if you have nested `setTimeout` calls?

Answer: Each `setTimeout` is registered independently with its own timer.

Example:

```
console.log("Start");

setTimeout(() => {
  console.log("Outer timeout - 2s");

  setTimeout(() => {
    console.log("Inner timeout - 1s");
  }, 1000);

  setTimeout(() => {
    console.log("Another inner timeout - 500ms");
  }, 500);
}, 2000);

setTimeout(() => {
  console.log("Parallel timeout - 1s");
}, 1000);

console.log("End");
```

Execution Timeline:

At 0ms:

Output: Start

Output: End

Web API: Timer1 (2000ms) registered

Web API: Timer2 (1000ms) registered

At 1000ms:

Timer2 completes → Callback Queue

Output: Parallel timeout - 1s

At 2000ms:

Timer1 completes → Callback Queue

Output: Outer timeout - 2s

Web API: Timer3 (1000ms) registered ($2000 + 1000 = 3000\text{ms}$ total)

Web API: Timer4 (500ms) registered ($2000 + 500 = 2500\text{ms}$ total)

At 2500ms:

Timer4 completes

Output: Another inner timeout - 500ms

At 3000ms:

Timer3 completes

Output: Inner timeout - 1s

Complete Output:

Start

End

Parallel timeout - 1s

Outer timeout - 2s

Another inner timeout - 500ms

Inner timeout - 1s

Key Points:

1. **Nested timers don't start until their parent executes**
2. **Each timer is independent** in Web API
3. **Timers add up**: Inner timeout of 1s after outer 2s = 3s total
4. **All go to Callback Queue** when complete
5. **No blocking**: Other code continues while timers count

Common Use Case - Delayed Chain:

```
function delayedSequence() {
  setTimeout(() => {
    console.log("Step 1");
    setTimeout(() => {
      console.log("Step 2");
      setTimeout(() => {
        console.log("Step 3");
      }, 1000);
    }, 1000);
  }, 1000);
}
```

This creates a sequence: Step 1 (1s) → Step 2 (2s) → Step 3 (3s)

Note: This pattern is called "Callback Hell" and is why Promises were invented!

Question 3: Can you have a callback in Microtask Queue that adds another callback to Microtask Queue?

Answer: YES! And this can create interesting (sometimes problematic) scenarios.

Example 1: Microtask Adding Microtask

```
console.log("Start");

Promise.resolve().then(() => {
  console.log("Promise 1");

  Promise.resolve().then(() => {
    console.log("Promise 2 (nested)");
  });
});

console.log("End");
```

Execution Flow:

Step 1: Synchronous code

Output: Start

Promise 1 callback → Microtask Queue

Output: End

Call Stack: Empty

Step 2: Event Loop checks Microtask Queue

Move Promise 1 callback to Call Stack

Execute: Output "Promise 1"

Inside Promise 1: Promise 2 callback → Microtask Queue

Promise 1 completes, removed from stack

Step 3: Event Loop checks Microtask Queue AGAIN

Microtask Queue still has Promise 2

Move Promise 2 to Call Stack

Execute: Output "Promise 2 (nested)"

Output:

Start

End

Promise 1

Promise 2 (nested)

Example 2: Infinite Microtask Loop (Dangerous!)

```
console.log("Start");
```

```
function addMicrotask() {  
  Promise.resolve().then(() => {  
    console.log("Microtask executed");  
    addMicrotask(); // Adds another microtask!  
  });  
}
```

```
addMicrotask();
```

```
setTimeout(() => {  
  console.log("Timeout - Will this ever run?");  
}, 0);
```

```
console.log("End");
```

What Happens:

Start
End
Microtask executed
Microtask executed
Microtask executed
Microtask executed
... (infinite loop!)
(Timeout NEVER executes!)

Why Timeout Never Runs:

The Rule: Event Loop must empty Microtask Queue before checking Callback Queue

The Problem:

1. Microtask executes
2. Adds another microtask
3. Event Loop sees Microtask Queue is not empty
4. Executes new microtask
5. Which adds another microtask
6. Repeat forever
7. **Never reaches Callback Queue** where setTimeout is waiting

Practical Example - Microtask Chain:

```
console.log("Start");
```

```
Promise.resolve()  
  .then(() => {  
    console.log("Step 1");  
    return Promise.resolve();  
  })  
  .then(() => {  
    console.log("Step 2");  
    return Promise.resolve();  
  })  
  .then(() => {  
    console.log("Step 3");  
  });
```

```
setTimeout(() => {  
  console.log("Timeout");  
}, 0);
```

```
console.log("End");
```

Output:

Start
End
Step 1
Step 2
Step 3
Timeout

All promise handlers execute before `setTimeout` because they're all microtasks.

The Starvation Problem:

If microtasks keep adding microtasks, **macrotasks (`setTimeout`, events) will starve** - they'll never get executed!

```
// This will freeze your UI
let count = 0;
function recursiveMicrotask() {
  Promise.resolve().then(() => {
    count++;
    console.log(count);
    if (count < 1000000) {
      recursiveMicrotask(); // Keep adding microtasks
    }
  });
}
recursiveMicrotask();
// Click events, setTimeout callbacks won't work until this completes!
```

Question 4: What's the difference between `setTimeout(fn, 0)` and calling `fn()` directly?

Answer: Huge difference in timing and execution order!

Direct Call:

```
console.log("Start");

function myFunction() {
  console.log("Function executed");
}
```

```
}
```

```
myFunction(); // Direct call
```

```
console.log("End");
```

Output:

Start

Function executed

End

Executes immediately, synchronously, in order.

setTimeout with 0:

```
console.log("Start");
```

```
function myFunction() {  
  console.log("Function executed");  
}
```

```
setTimeout(myFunction, 0); // Async, even with 0ms!
```

```
console.log("End");
```

Output:

Start

End

Function executed

Why the difference?

Execution Flow with setTimeout(fn, 0):

1. **"Start" prints** (synchronous)
2. **setTimeout encountered:**
 - JavaScript: "This is async"
 - Passes to Web API
 - Web API registers timer (0ms)
 - JavaScript continues immediately

3. **"End" prints** (synchronous)
4. **Call Stack empties**
5. **Web API timer completes** (0ms passed)
6. **Callback added to Callback Queue**
7. **Event Loop**: Stack empty? ✓ Queue has callback? ✓
8. **Moves callback to stack**
9. **"Function executed" prints**

Key Differences:

Aspect	Direct Call <code>fn()</code>	<code>setTimeout(fn, 0)</code>
Execution	Immediate	After current code completes
Queue	No queue	Goes to Callback Queue
Blocks	Yes (if function is heavy)	No (registers and continues)
Order	Executes in sequence	Executes after synchronous code
Timing	Exact timing	At least 0ms, usually 1-4ms
Call Stack	Adds to stack immediately	Waits for stack to empty

Practical Use Cases:

Use Case 1: Deferring Execution

```
function processLargeData() {
  // Heavy computation
  for(let i = 0; i < 1000000; i++) {
    // Process
  }
}

// Without setTimeout - blocks UI
console.log("Before");
processLargeData(); // UI freezes here
console.log("After"); // Waits for above to complete

// With setTimeout - non-blocking
console.log("Before");
setTimeout(processLargeData, 0); // Deferred
console.log("After"); // Executes immediately
// UI stays responsive
```

Use Case 2: Breaking Up Long Tasks

```
function processBatch(items, batchSize, callback) {
  let index = 0;

  function processPart() {
    const endIndex = Math.min(index + batchSize, items.length);

    for (let i = index; i < endIndex; i++) {
      // Process item
    }

    index = endIndex;

    if (index < items.length) {
      setTimeout(processPart, 0); // Give UI a chance to breathe
    } else {
      callback();
    }
  }

  processPart();
}
```

Use Case 3: Ensuring DOM is Ready

```
element.textContent = "Updated";

// Want to get the updated height
setTimeout(() => {
  const height = element.offsetHeight; // Gets accurate height
}, 0);
```

Minimum Delay Truth:

Even `setTimeout(fn, 0)` has a **minimum delay**:

- **Browsers:** Usually 1-4ms minimum
- **HTML5 Spec:** 4ms minimum for nested timers
- **Not truly 0ms:** Just "as soon as possible"

Example Showing Real Delay:

```
const start = performance.now();
```

```
setTimeout(() => {
  const end = performance.now();
  console.log(`Actual delay: ${end - start}ms`);
}, 0);
```

// Output: Actual delay: 1.5ms (or similar, never exactly 0)

Question 5: How would you make synchronous-looking code using async operations?

Answer: Use **async/await** syntax! It makes asynchronous code look synchronous while maintaining non-blocking behavior.

Traditional Callback Approach (Callback Hell):

```
function getUserData() {
  setTimeout(() => {
    console.log("Got user");
    setTimeout(() => {
      console.log("Got posts");
      setTimeout(() => {
        console.log("Got comments");
        setTimeout(() => {
          console.log("All done!");
        }, 1000);
      }, 1000);
    }, 1000);
  }, 1000);
}
```

Problems: Hard to read, maintain, debug (Pyramid of Doom!)

Promise Approach (Better):

```
function getUser() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("User data");
    }, 1000);
  });
}
```

```
}

function getPosts() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Posts data");
    }, 1000);
  });
}
```

```
function getComments() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Comments data");
    }, 1000);
  });
}
```

```
// Using Promises
getUser()
  .then(user => {
    console.log(user);
    return getPosts();
  })
  .then(posts => {
    console.log(posts);
    return getComments();
  })
  .then(comments => {
    console.log(comments);
  });
```

Better: Chainable, but still not as clean as sync code.

Async/Await Approach (Best):

```
async function fetchAllData() {
  console.log("Start fetching...");

  const user = await getUser();
  console.log(user);

  const posts = await getPosts();
```

```
    console.log(posts);

    const comments = await getComments();
    console.log(comments);

    console.log("All done!");
}

fetchAllData();
console.log("This runs immediately!");
```

Output:

Start fetching...
This runs immediately!
(1 second later) User data
(1 second later) Posts data
(1 second later) Comments data
All done!

Looks synchronous, but it's not blocking!

How It Works Under the Hood:

```
async function example() {
    console.log("1");
    const result = await someAsyncOperation();
    console.log("2");
    console.log(result);
}

example();
console.log("3");
```

What JavaScript Does:

1. Enters `example()` function
2. Executes `console.log("1")`
3. Hits `await` keyword
4. **Pauses function execution**
5. **Returns control to main thread**

6. Executes `console.log("3")`
7. When promise resolves, **resumes function**
8. Continues with `console.log("2")` and `console.log(result)`

Output:

1
3
2
[result value]

Practical Example: Fetching Data

Without Async/Await (Callback):

```
fetch('https://api.github.com/users')
  .then(response => response.json())
  .then(users => {
    console.log(users[0].login);
    fetch(`https://api.github.com/users/${users[0].login}`)
      .then(response => response.json())
      .then(user => {
        console.log(user.name);
        fetch(user.repos_url)
          .then(response => response.json())
          .then(repos => {
            console.log(repos.length);
          });
      });
  });
```

Messy and hard to follow!

With Async/Await (Clean):

```
async function getUserInfo() {
  try {
    // Get all users
    const usersResponse = await fetch('https://api.github.com/users');
    const users = await usersResponse.json();
    console.log(users[0].login);
  }
}
```

```

    // Get specific user
    const userResponse = await fetch(`https://api.github.com/users/${users[0].login}`);
    const user = await userResponse.json();
    console.log(user.name);

    // Get user's repos
    const reposResponse = await fetch(user.repos_url);
    const repos = await reposResponse.json();
    console.log(repos.length);

    return { user, repos };
  } catch (error) {
    console.error("Error:", error);
  }
}

getUserInfo();

```

Reads like synchronous code but doesn't block!

Advanced Pattern: Parallel Execution

Sequential (Slow):

```

async function sequential() {
  const user1 = await fetchUser(1); // Wait 1s
  const user2 = await fetchUser(2); // Wait 1s
  const user3 = await fetchUser(3); // Wait 1s
  // Total: 3 seconds
  return [user1, user2, user3];
}

```

Parallel (Fast):

```

async function parallel() {
  // Start all at once
  const promise1 = fetchUser(1);
  const promise2 = fetchUser(2);
  const promise3 = fetchUser(3);
}

```

```

// Wait for all to complete
const [user1, user2, user3] = await Promise.all([
  promise1,
  promise2,
  promise3
]);

// Total: 1 second (all run in parallel)
return [user1, user2, user3];
}

```

Or more concisely:

```

async function parallel() {
  const users = await Promise.all([
    fetchUser(1),
    fetchUser(2),
    fetchUser(3)
  ]);
  return users;
}

```

Error Handling - Synchronous Style:

```

async function handleErrors() {
  try {
    const response = await fetch('https://api.example.com/data');

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const data = await response.json();
    return data;

  } catch (error) {
    console.error("Failed to fetch:", error);
    return null;
  }
}

```

Uses familiar `try/catch` blocks instead of `.catch()` chains!

The Magic of Async/Await:

Benefits:

1.  **Reads like synchronous code** - easier to understand
2.  **Better error handling** - use try/catch
3.  **Easier debugging** - step through with debugger
4.  **Clean conditionals** - no nested `.then()` chains
5.  **Still non-blocking** - doesn't freeze UI

Under the Hood:

- `async` function always returns a Promise
- `await` pauses function execution (but not the thread!)
- When promise resolves, function resumes
- Everything after `await` is like a `.then()` callback

Example:

// These are equivalent:

// Async/await version

```
async function example() {  
  const result = await somePromise();  
  console.log(result);  
}
```

// Promise version (what it actually does)

```
function example() {  
  return somePromise().then(result => {  
    console.log(result);  
  });  
}
```

Complete Real-World Example:

// Simulated API functions

```
function fetchUser(id) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve({ id, name: `User${id}`, email: `user${id}@example.com` });  
    }, 1000);  
  });  
}
```

```
});  
}
```

```
function fetchPosts(userId) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve([  
        { id: 1, title: "Post 1", userId },  
        { id: 2, title: "Post 2", userId }  
      ]);  
    }, 1000);  
  });  
}
```

```
function fetchComments(postId) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve([  
        { id: 1, text: "Comment 1", postId },  
        { id: 2, text: "Comment 2", postId }  
      ]);  
    }, 1000);  
  });  
}
```

// Clean, readable async code

```
async function loadUserDashboard(userId) {  
  try {  
    console.log("Loading dashboard...");  
  
    // Get user info  
    const user = await fetchUser(userId);  
    console.log(`Welcome, ${user.name}!`);  
  
    // Get user's posts  
    const posts = await fetchPosts(userId);  
    console.log(`You have ${posts.length} posts`);  
  
    // Get comments for first post  
    if (posts.length > 0) {  
      const comments = await fetchComments(posts[0].id);  
      console.log(`First post has ${comments.length} comments`);  
    }  
  }  
}
```

```
    return { user, posts };

  } catch (error) {
    console.error("Failed to load dashboard:", error);
    throw error;
  }
}

// Usage
loadUserDashboard(1)
  .then(data => console.log("Dashboard loaded:", data))
  .catch(err => console.error("Error:", err));

console.log("This runs immediately while dashboard loads!");
```

Output:

```
Loading dashboard...
This runs immediately while dashboard loads!
(1s later) Welcome, User1!
(1s later) You have 2 posts
(1s later) First post has 2 comments
Dashboard loaded: { user: {...}, posts: [...] }
```

Summary: Async/await is syntactic sugar that makes asynchronous code look and behave like synchronous code, while maintaining all the benefits of non-blocking execution!

Final Thoughts

The beauty of JavaScript's execution model is:

- **Simple to understand** once you know the rules
- **Powerful for building responsive UIs**
- **Non-blocking by design** through delegation
- **Predictable** through Event Loop priorities

Remember the golden rules:

1. JavaScript is single-threaded and synchronous
2. Browser/Web APIs provide async capabilities

3. Microtasks always run before macrotasks
4. Event Loop coordinates everything
5. Async/await makes it all look beautiful!

Remember: JavaScript never waits. It delegates, continues, and comes back later.